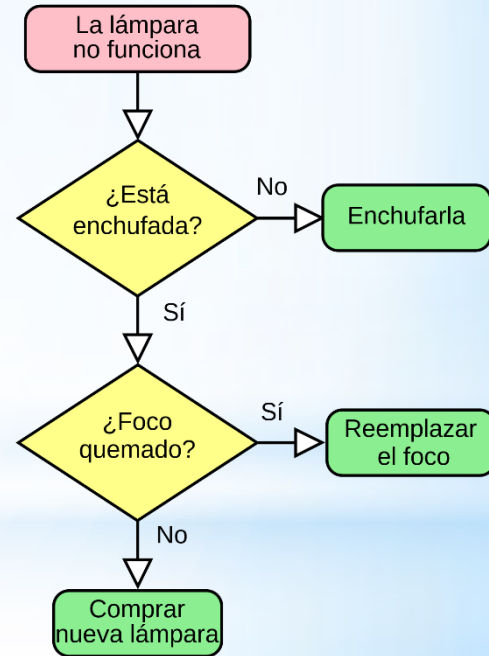


ALGORITMO

Descripción de un proceso paso a paso que permite llegar a la solución de un problema

Suma de dos números

1. Ingresar el valor de los dos números (a y b)
2. Realizar la operación:
 $c = a + b$
3. Visualizar el resultado

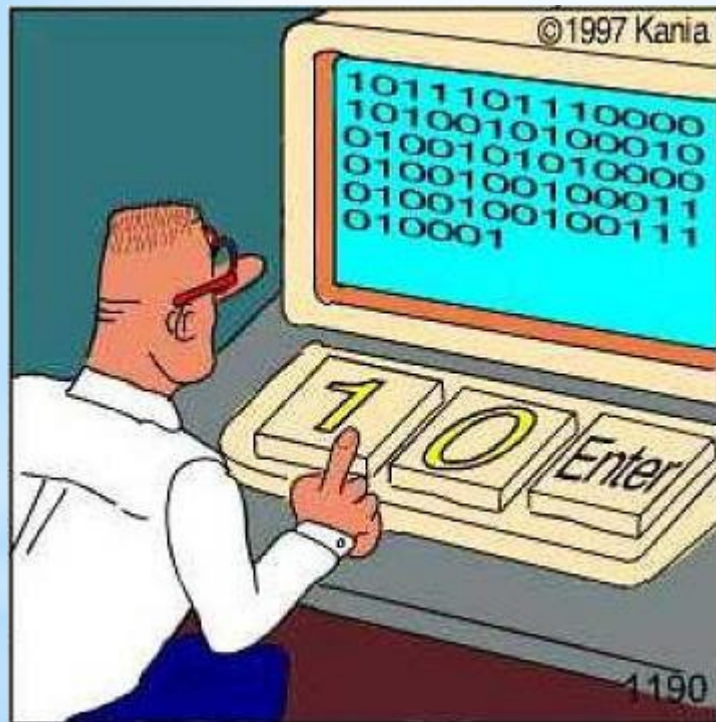


PROGRAMA

Descripción para la computadora de un proceso que obtiene cierto resultado útil para alguien



COMUNICACIÓN



```
PROGRAM
VAR
    TIMER1 : TON;
    TIMER2 : TON;
    TAG-1 : BOOL;
    TAG-2 : INT;
    TIN : TIME;
    TOUT : TIME;
END_VAR
```

```
-----
LD TAG-1
ST TIMER1.IN
GOTO MARK1
CAL TIMER1(
    PT:TIN1,
    ET:>TOUT1)
LD TIMER1.Q
ST TIMER2.IN
-----
```

```
MARK1:
LD TAG-2
AND 130
OR 3
```

LEARN ROBOTICS

INSTRUCTION LIST

- ✓ Best for compact code
- ✓ Great for Time Critical code
- ✓ One Command per Line
- ✗ More difficult to resolve errors

LENGUAJE DE PROGRAMACIÓN

Permite expresar una serie de instrucciones que podrán ser realizadas por una computadora



ELEMENTOS DE UN LENGUAJE

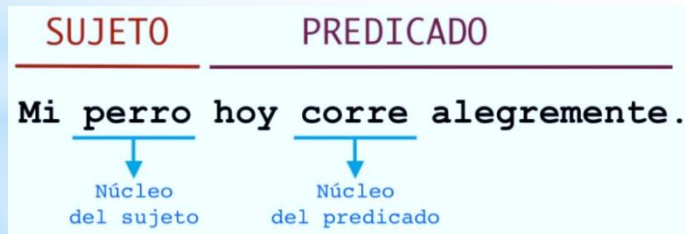
Alfabeto



Léxico



Sintaxis



Semántica

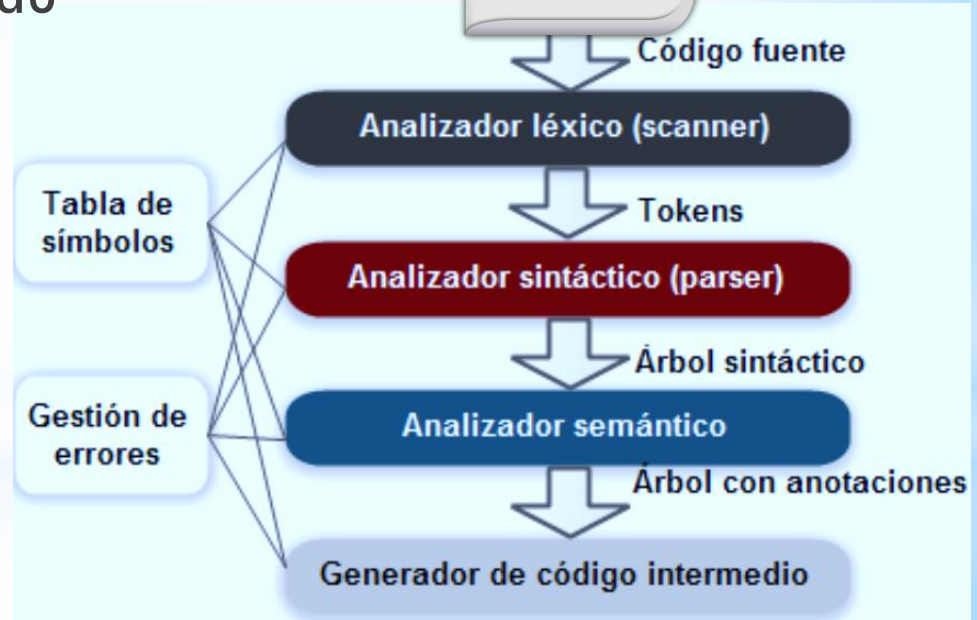


PROGRAMACIÓN

Componer los elementos de un lenguaje de programación en el orden que provocará un efecto deseado



```
>>> len?
Traceback (most recent call last):
  Python Shell, prompt 13, line 1
SyntaxError: len?: <string>, line 1, pos 4
>>> len('hello')
Traceback (most recent call last):
  Python Shell, prompt 14, line 1
SyntaxError: len'hello': <string>, line 1, pos 10
>>> len(hello)
Traceback (most recent call last):
  Python Shell, prompt 15, line 1
builtins.NameError: name 'hello' is not defined
>>>
```



NIVELES LENGUAJES DE PROGRAMACIÓN

Alto Nivel

```
#!/usr/bin/env python3
# Autor:          Angel Brito Segura
# Fecha de creación: 21/06/2023
# Descripción:    Suma de dos números enteros

def add(num1, num2):
    return num1 + num2

x = int(input("Escribe el primer sumando: "))
y = int(input("Escribe el segundo sumando: "))
print("La suma de los dos números es", add(x,y))
```

Mediano Nivel

```
#include <stdio.h>
int add ( int x, int y );
int main()
{
    int x;
    int y;
    printf("Escribe los dos números que se sumarán: ");
    scanf("%d", &x );
    scanf("%d", &y );
    printf("La suma de los números es %d\n" add( x, y ));
    getchar();
}
int add ( int x, int y )
{
    return x + y;
}
```

Bajo Nivel

```
PROCESSOR 16f877 ;Indica versión del procesador (PIC16F877)
INCLUDE <pic16f877.inc> ;Librería de la versión del procesador PIC16F877

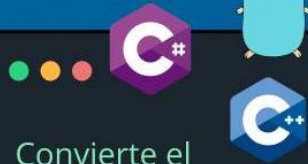
K equ H'26' ;Variable K en la dirección de memoria 26 hexadecimal
L equ H'27' ;Variable L en la dirección de memoria 27 hexadecimal
M equ H'28' ;Variable M en la dirección de memoria 28 hexadecimal

ORG 0 ;Especifica el vector de reset inicie en 0
GOTO INICIO ;Ir a la dirección de memoria que se encuentra INICIO
ORG 5 ;Indica origen para inicio de programa

INICIO: MOVF K,W ;Mover lo que tenga K al registro W
        ADDWF L,0 ;Sumar L con lo que tenga W
        MOVWF M ;Mover lo que tenga W a dirección de memoria de L
        GOTO INICIO ;Ir a la dirección de memoria que se encuentra INICIO
END ;Termina el programa
```

TIPOS LENGUAJES DE PROGRAMACIÓN

COMPILADO



Convierte el código a binarios que lee el sistema operativo.



INTERPRETADO



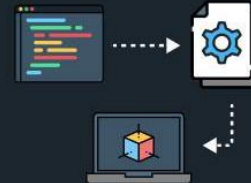
Requiere de un programa que lea la instrucción del código en tiempo real, y la ejecute.



INTERMEDIO

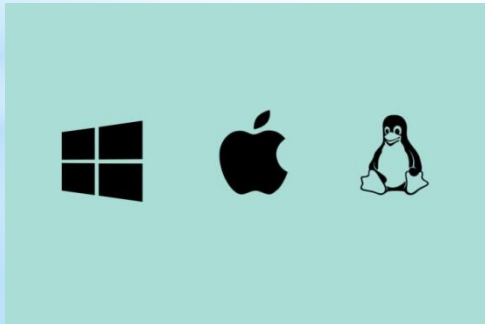


Se compila el código fuente a un lenguaje intermedio y este último se ejecuta en una máquina virtual.

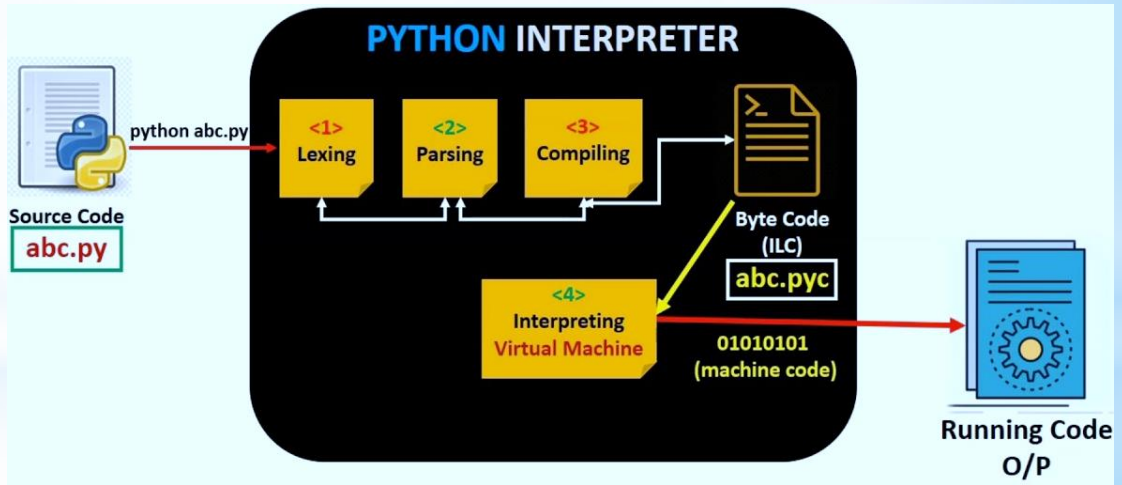


PYTHON

- * Lenguaje de programación de alto nivel interpretado o de script
- * Multiparadigma
- * Tipado dinámico
- * Fuertemente tipado
- * Multiplataforma



INTÉRPRETE

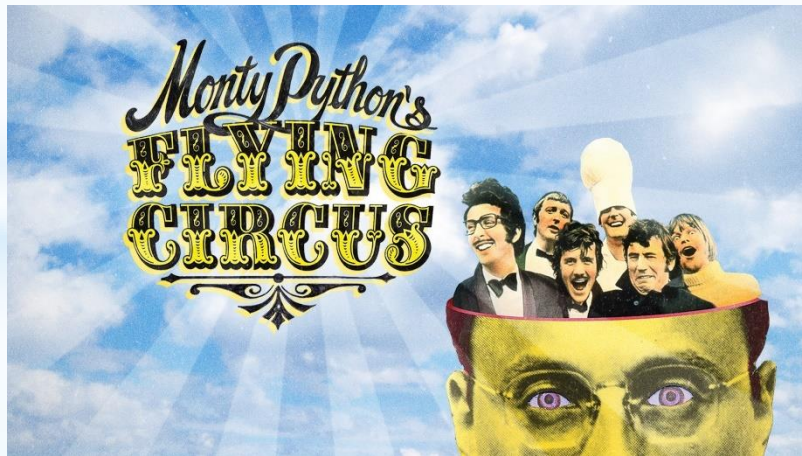


OBJETIVOS

- * Un lenguaje fácil e intuitivo
- * Código abierto
- * Código tan comprensible como el inglés
- * Adecuado para tareas cotidianas, permitiendo tiempos de desarrollo cortos



Guido van Rossum



VERSIONES

PYTHON 2.X



PYTHON 3.X

← **LEGACY**

It is still entrenched in the software at certain companies



LIBRARY

Many older libraries built for Python 2 are not forwards compatible

0100
0001 **ASCII**

Strings are stored as ASCII by default

≈ **7/2=3**

It rounds your calculation down to the nearest whole number



```
print "WELCOME TO  
GEEKSFORGEEKS"
```

FUTURE →

It will take over Python 2 by the end of 2019

LIBRARY



Many of today's developers are creating libraries strictly for use with Python 3

UNICODE
0000
0000
0100
0001

Text Strings are Unicode by default

7/2=3.5 =

This expression will result in the expected result

```
print("WELCOME TO  
GEEKSFORGEEKS")
```



Python Implementations

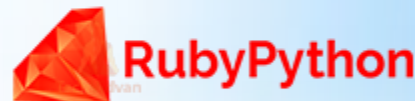


python SOFTWARE FOUNDATION

IronPython



MicroPython



APLICACIONES



```
import flet
from flet import IconButton, Page, Row, TextField, icons

def main(page: Page):
    page.title = "Flet counter example"
    page.vertical_alignment = "center"

    txt_number = TextField(value="0", text_align="right",
                           width=100)

    def minus_click(e):
        txt_number.value = int(txt_number.value) - 1
        page.update()

    def plus_click(e):
        txt_number.value = int(txt_number.value) + 1
        page.update()

    page.add(
        Row(
            [
                IconButton(icons.REMOVE, on_click=minus_click),
                txt_number,
                IconButton(icons.ADD, on_click=plus_click),
            ],
            alignment="center"
        )
    )

flet.app(target=main)
```



Write Flutter Apps with Python



```
from reactpy import component, html, run

@component
def Button():
    def handle_event(event):
        print(event)

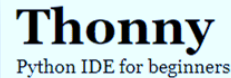
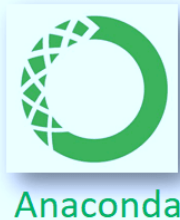
    return html.button({"on_click": handle_event}, "Click me!")

run(Button)
```



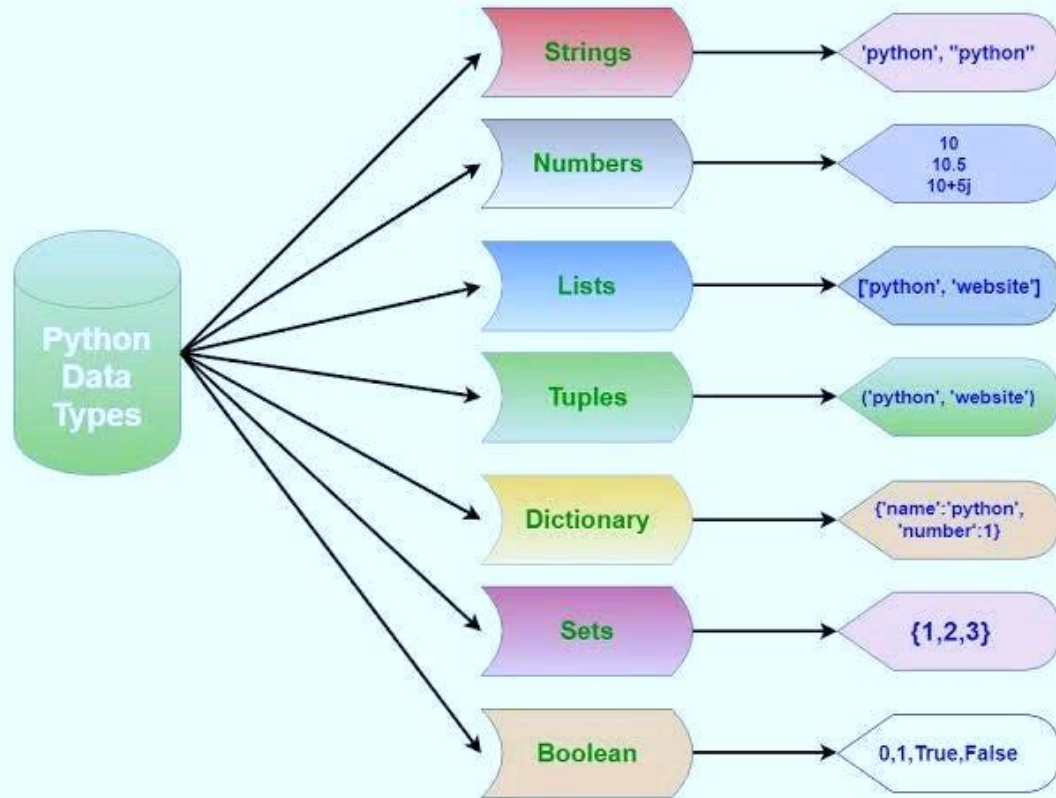
ENTORNOS DE PROGRAMACIÓN

- * Modo Interactivo (consola)
- * Interpretando código (scripts)
- * Entornos de Desarrollo Integrado (IDE)

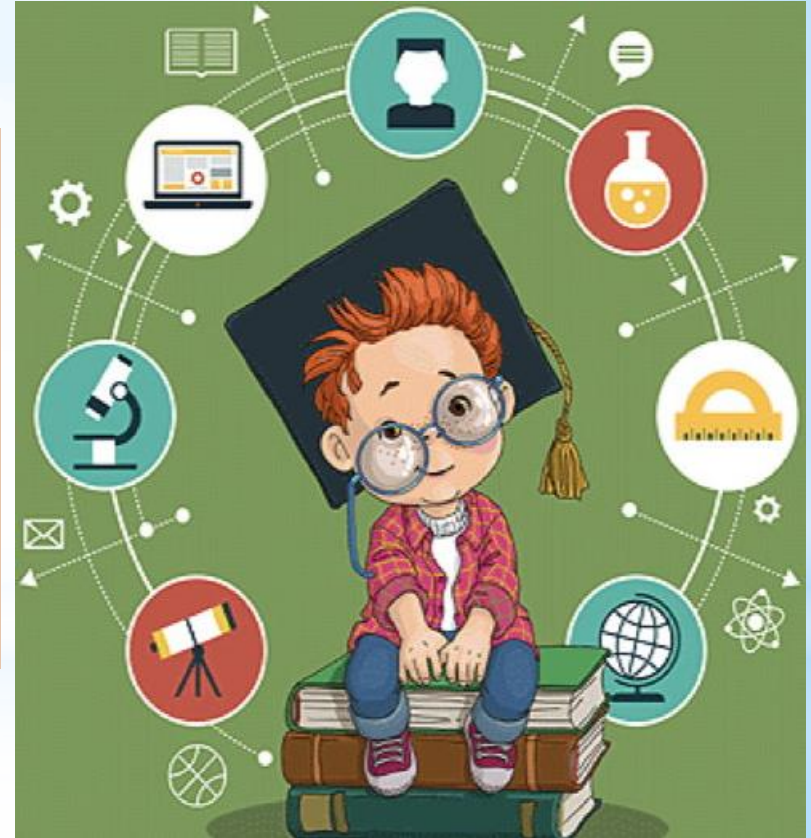
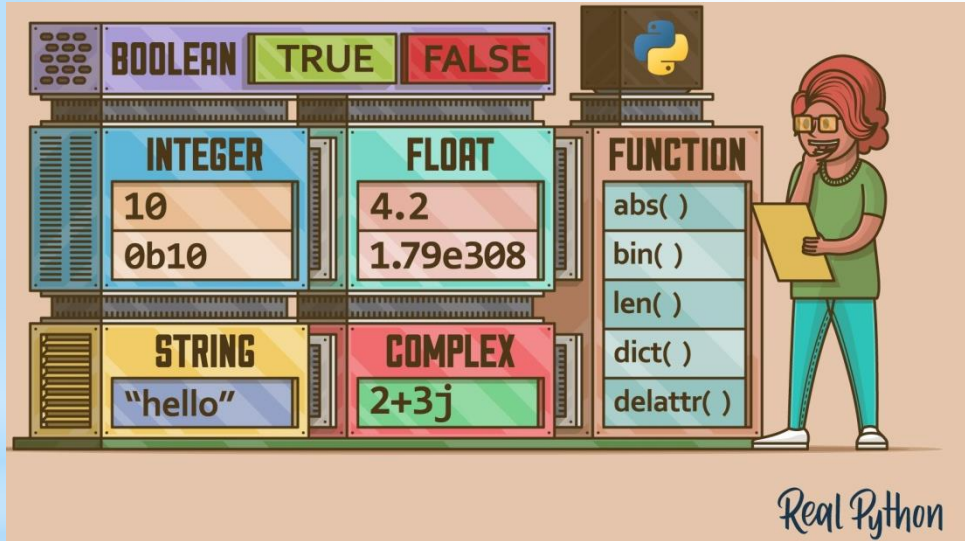


LITERAL

Notación para representar valores fijos



DATOS



TIPO DE DATO - NUMÉRICO



Decimal
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Binario
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

Octal
0 ₈
1 ₈
2 ₈
3 ₈
4 ₈
5 ₈
6 ₈
7 ₈
10 ₈
11 ₈
12 ₈
13 ₈
14 ₈
15 ₈
16 ₈
17 ₈

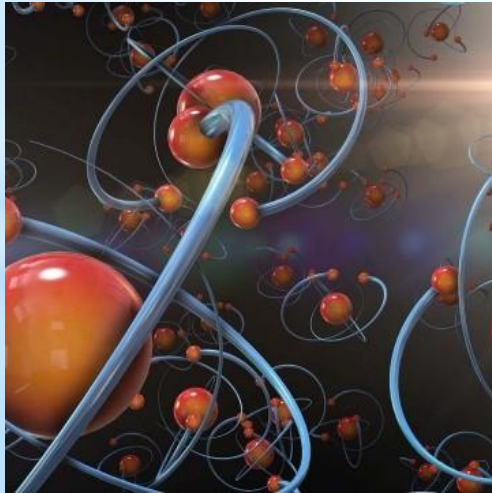
Hexadecimal
0
1
2
3
4
5
6
7
8
9
A
B
C
D
E
F

NOTACIÓN CIENTÍFICA

Representación sencilla de números muy pequeños/grandes

$$\boxed{a} \times \boxed{10}^{\boxed{n}}$$

Coeficiente Base Exponente



OPERADOR

Símbolo capaz de realizar operaciones con valores

expression

500

3.14

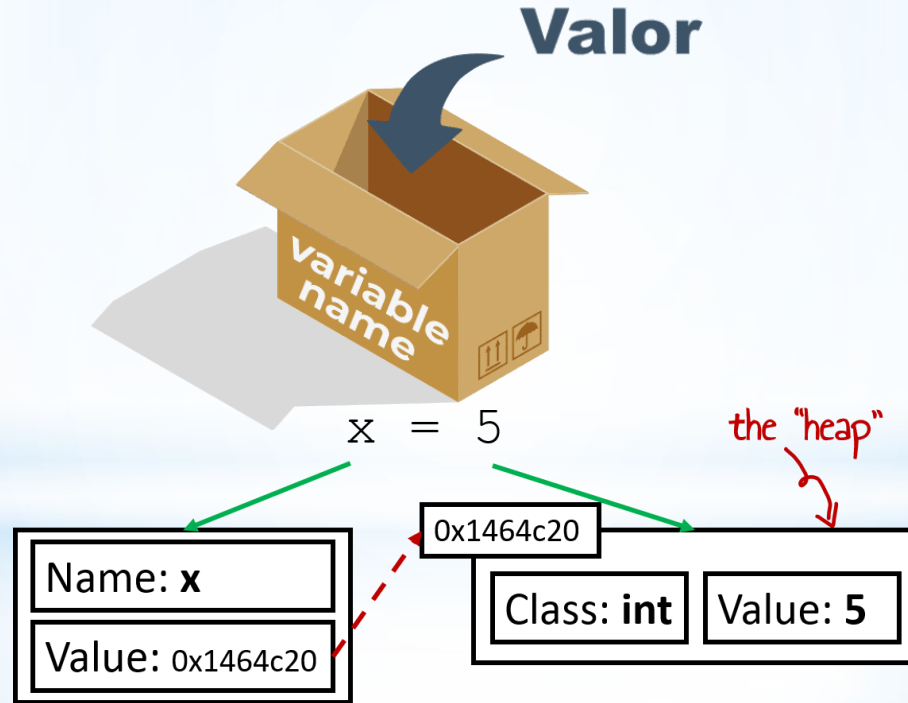
200 + 300

10.0 + 5.0

Prioridad	Operador											
1	~			+ (unario)				- (unario)				
2	**											
3	*			/				//		%		
4	+ (binario)						- (binario)					
5	<<						>>					
6	<			<=				>		>=		
7	==						!=					
8	&											
9												
10	=	+=	--	*=	/=	%=	&=	^=	=	>>=	<<=	

VARIABLE

Ubicación nombrada reservada para almacenar valores





-
- The image shows a 5x5 grid of Python keywords, each in a colored box. A large red 'X' is drawn over the entire grid. The keywords are:

Row 1: False, True, None, import, pass

Row 2: Name, break, else, in, raise

Row 3: Yes, class, finally, return

Row 4: and, continue, lambda, try

Row 5: as, def, from, nonlocal, while

Row 6: assert, del, global, with

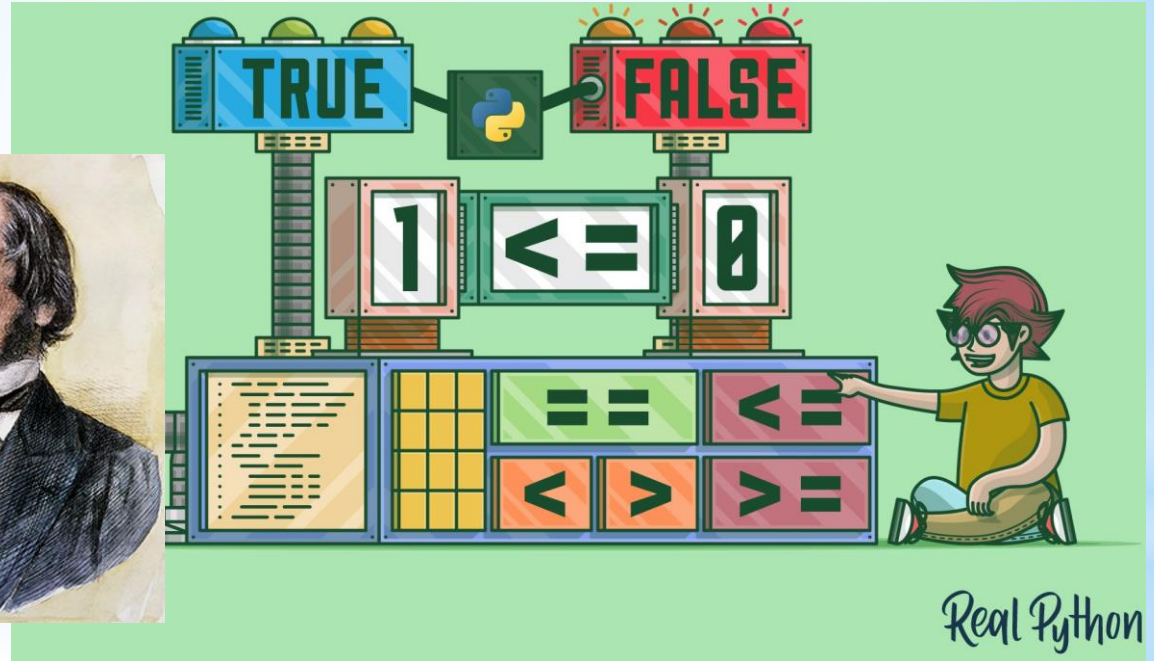
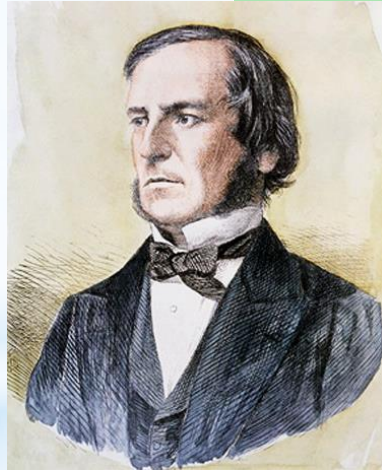
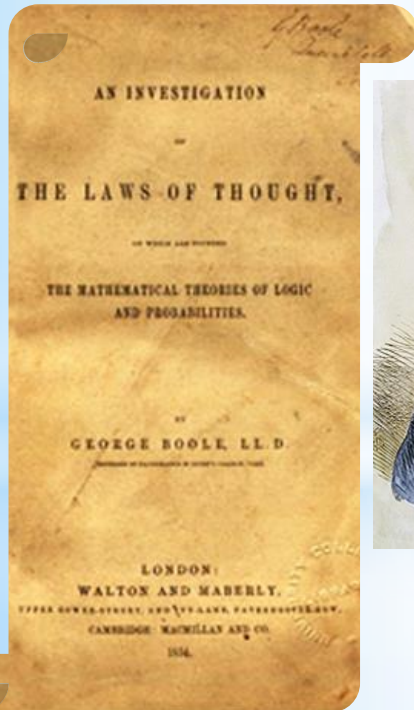
Row 7: async, elif, if, or, yield

Row 8: (empty), (empty), (empty), (empty), (empty)

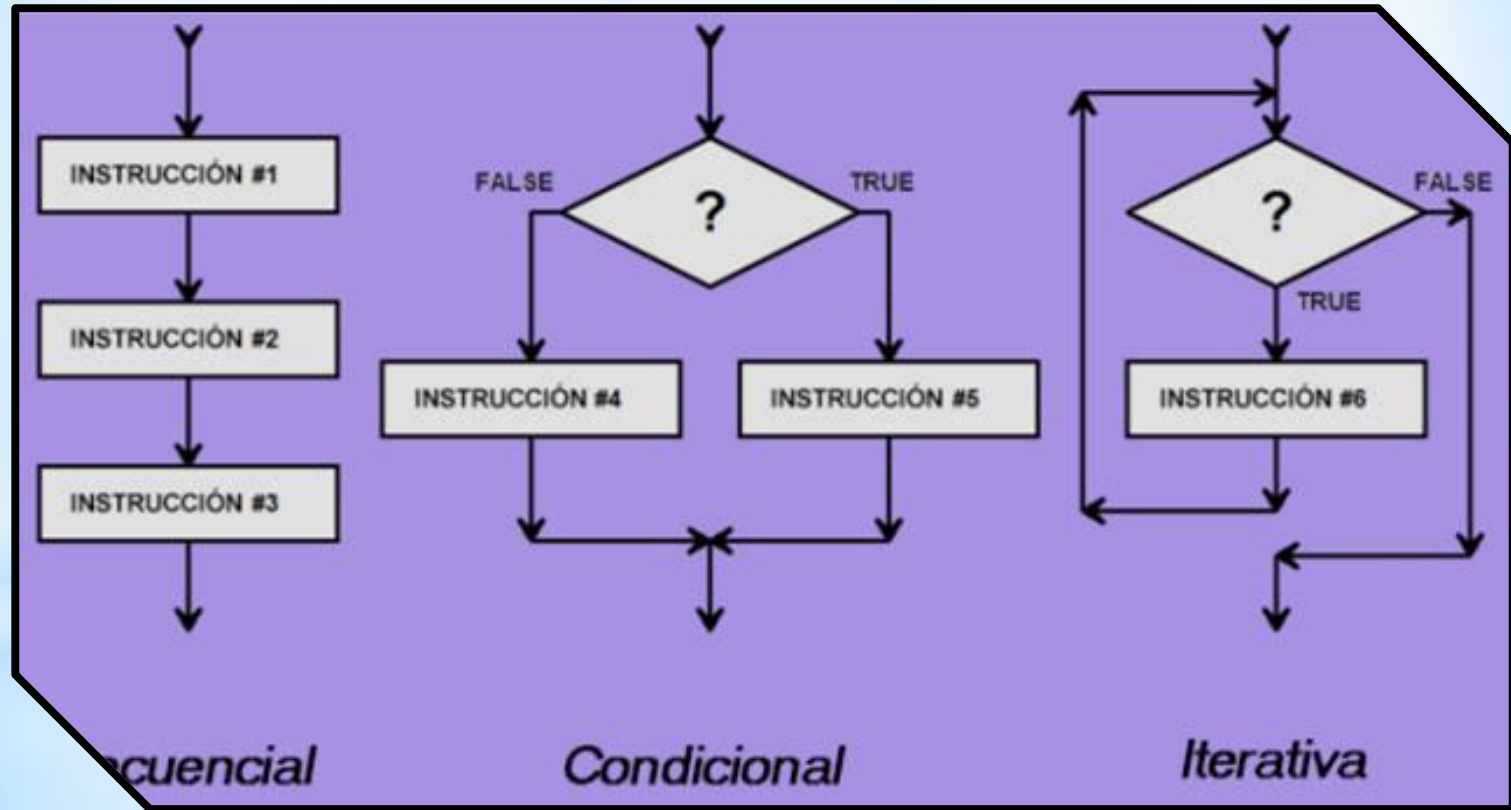
The man is holding a blue folder with a smiley face on it. In the background, there is a computer monitor displaying code and a keyboard.

TIPO DE DATO - BOOLEANOS

Se emplean para representar la veracidad



ESTRUCTURAS DE CONTROL



SENTENCIA CONDICIONAL

```
if <condición>:
```

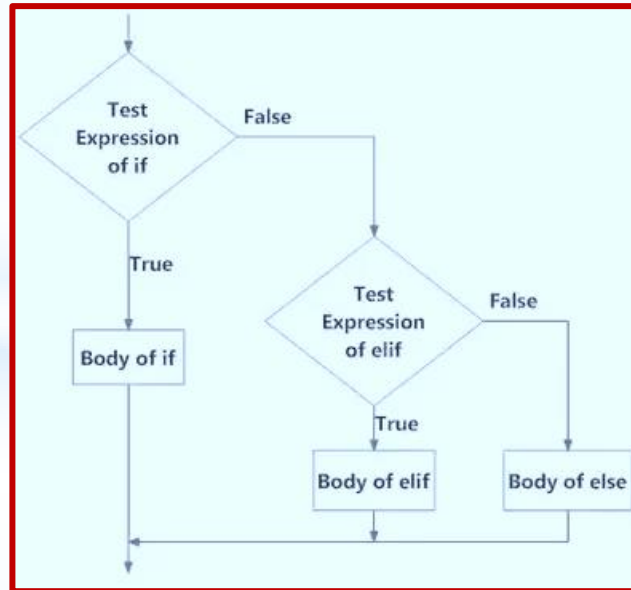
```
    #Instrucciones a ejecutar si la condición es cierta
```

```
elif <otra_condición>:
```

```
    #Instrucciones a ejecutar si se cumple otra condición
```

```
else:
```

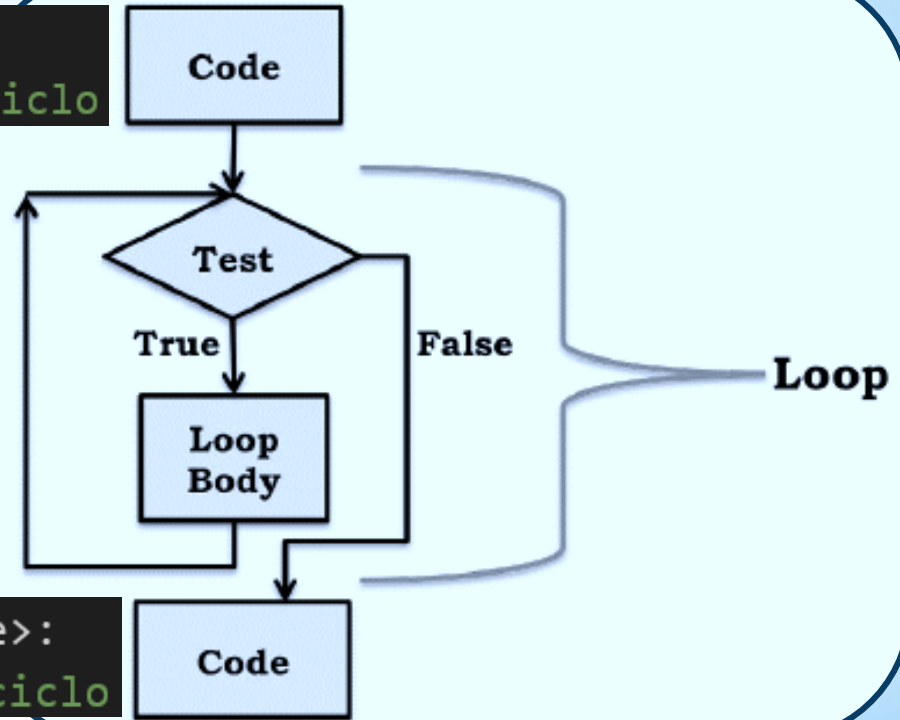
```
    #Instrucciones a ejecutar si no se cumple ninguna condición
```



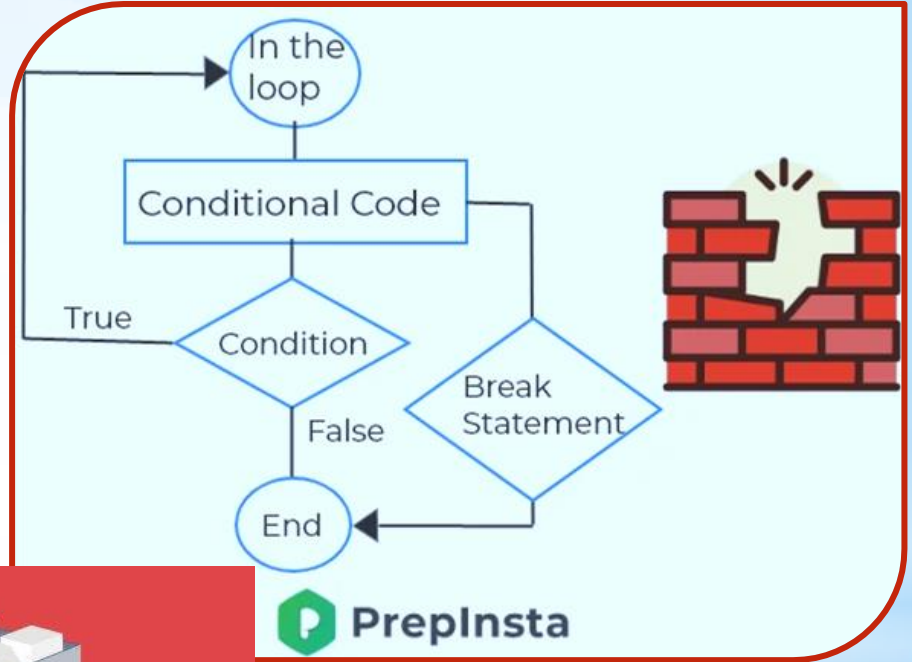
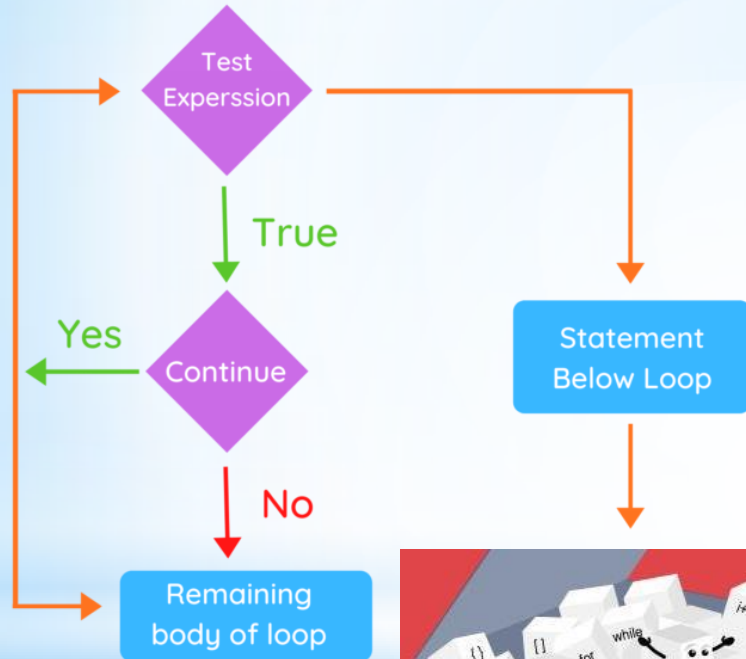
SENTENCIAS ITERATIVAS

```
while <expresión_condicional>:  
    #Instrucciones a ejecutar en el ciclo
```

```
for <variable> in <secuencia/iterable>:  
    #Instrucciones a realizar en el ciclo
```



BREAK & CONTINUE

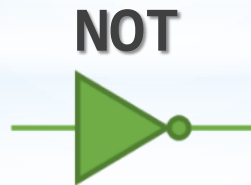


OPERADORES LÓGICOS

AND	True	False
True	True	False
False	False	False

OR	True	False
True	True	True
False	True	False

Q	Q'
0	1
1	0



$$Q = \bar{Q}$$

OPERADORES BIT A BIT

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

AND (&)

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

OR (|)

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

XOR (^)

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

NOT (~)

APLICACIÓN BITWISE OPERATIONS

TABLA DE VERDAD (XOR)

A	B	SALIDA
0	0	0
0	1	1
1	0	1
1	1	0



En OpenCV:  Visualización

A	B	SALIDA
0	0	0
0	255	255
255	0	255
255	255	0

A	B	SALIDA
		
		
		
		

OpenCV usa la función `cv2.bitwise_xor`, veamos a continuación un ejemplo aplicando esta función:



`cv2.bitwise_xor`



$$N * 2^m$$

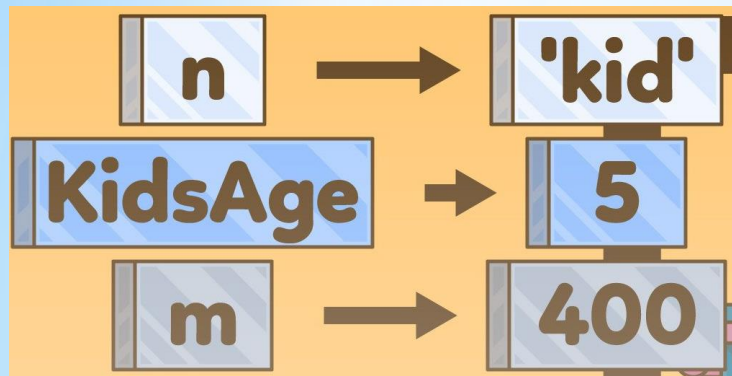


Left Shift (<<)

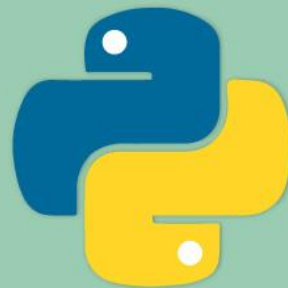
$$N // 2^m$$



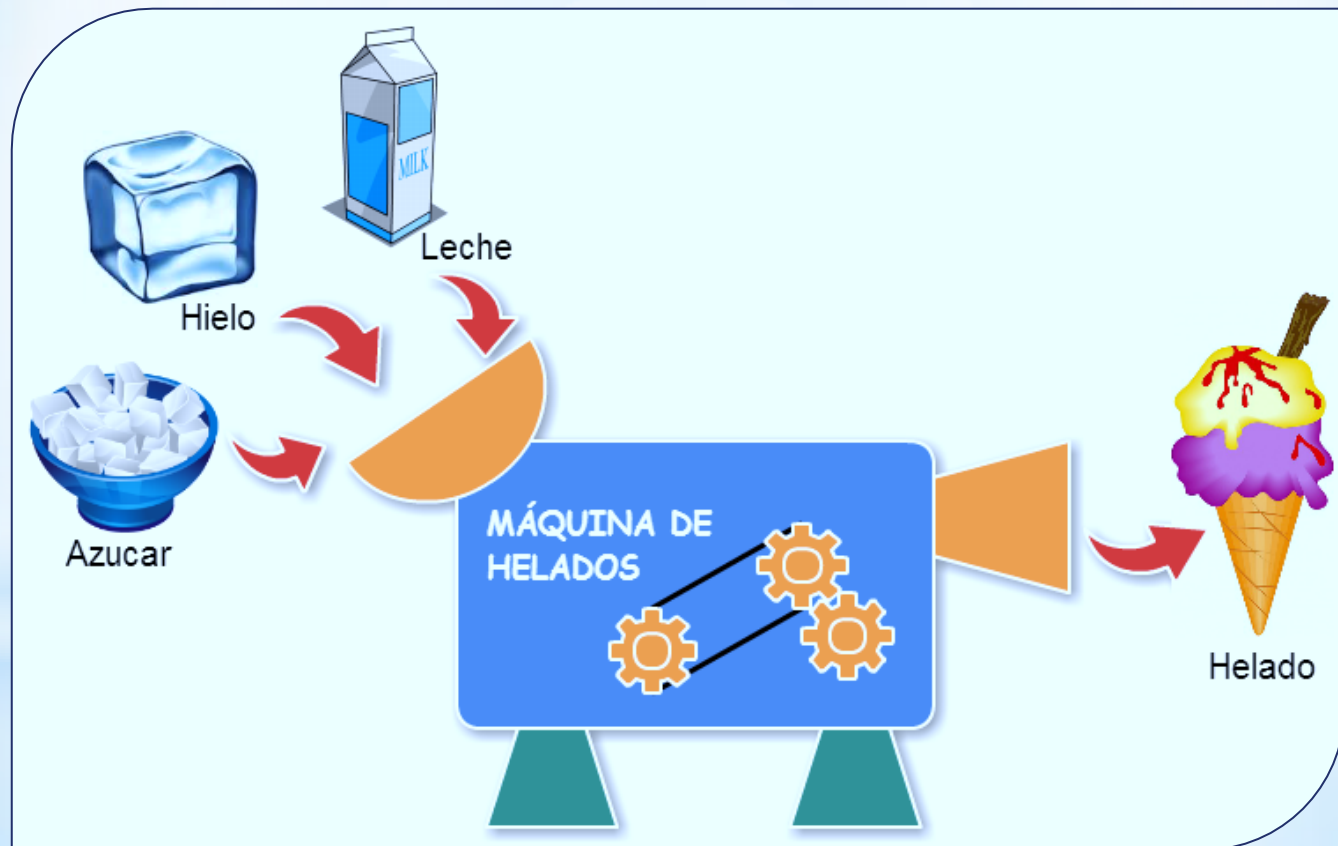
Right Shift (>>)



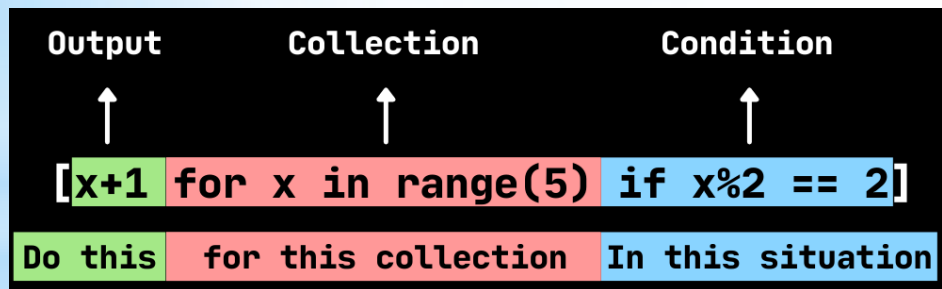
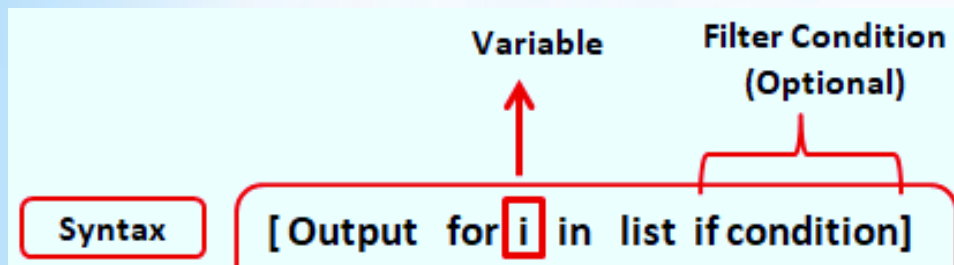
LISTAS con



FUNCIONES Y MÉTODOS



COMPRENSIÓN DE LISTAS



	A	B	C	D	E	F	G	H	
8	[0] [0]	[0] [1]	[0] [2]	[0] [3]	[0] [4]	[0] [5]	[0] [6]	[0] [7]	8
7	[1] [0]	[1] [1]	[1] [2]	[1] [3]	[1] [4]	[1] [5]	[1] [6]	[1] [7]	7
6	[2] [0]	[2] [1]	[2] [2]	[2] [3]	[2] [4]	[2] [5]	[2] [6]	[2] [7]	6
5	[3] [0]	[3] [1]	[3] [2]	[3] [3]	[3] [4]	[3] [5]	[3] [6]	[3] [7]	5
4	[4] [0]	[4] [1]	[4] [2]	[4] [3]	[4] [4]	[4] [5]	[4] [6]	[4] [7]	4
3	[5] [0]	[5] [1]	[5] [2]	[5] [3]	[5] [4]	[5] [5]	[5] [6]	[5] [7]	3
2	[6] [0]	[6] [1]	[6] [2]	[6] [3]	[6] [4]	[6] [5]	[6] [6]	[6] [7]	2
1	[7] [0]	[7] [1]	[7] [2]	[7] [3]	[7] [4]	[7] [5]	[7] [6]	[7] [7]	1
	A	B	C	D	E	F	G	H	

DICCIONARIOS Y TUPLAS



dictionaries

keys

'b'

'j'

'c'

values

'beauty'

'joy'

'computing'



lists

indices

0

1

2

values

'beauty'

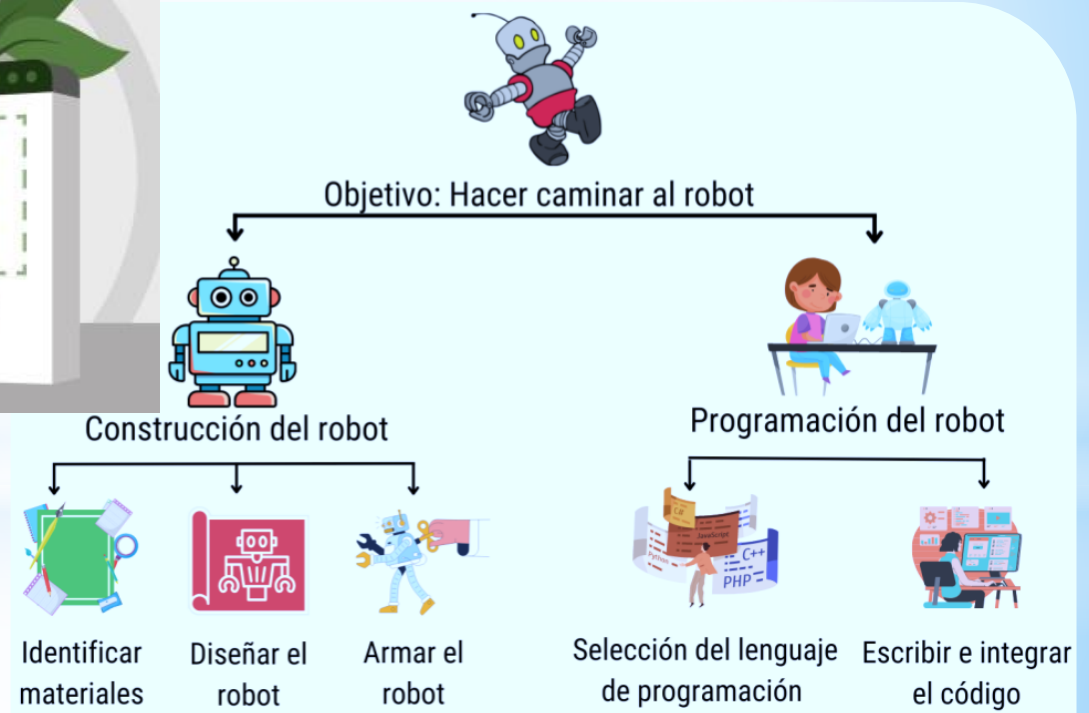
'joy'

'computing'

(⁰T₋₆, ¹U₋₅, ²P₋₄, ³L₋₃, ⁴A₋₂, ⁵S₋₁)

DESCOMPOSICIÓN

Dividir un problema en piezas aisladas



FUNCIONES

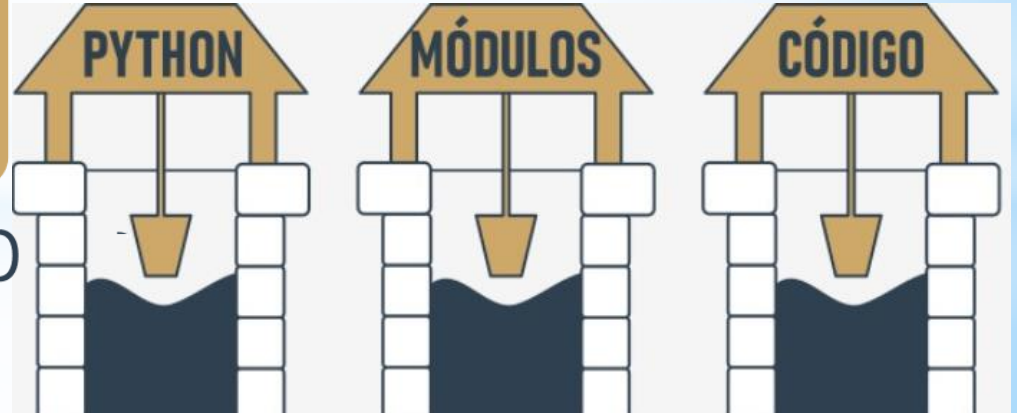
```
def <nombre_función>(<parametros>):  
    #Instrucciones a realizar
```

invocación

```
def message():  
    print("Ingresar valor: ")
```

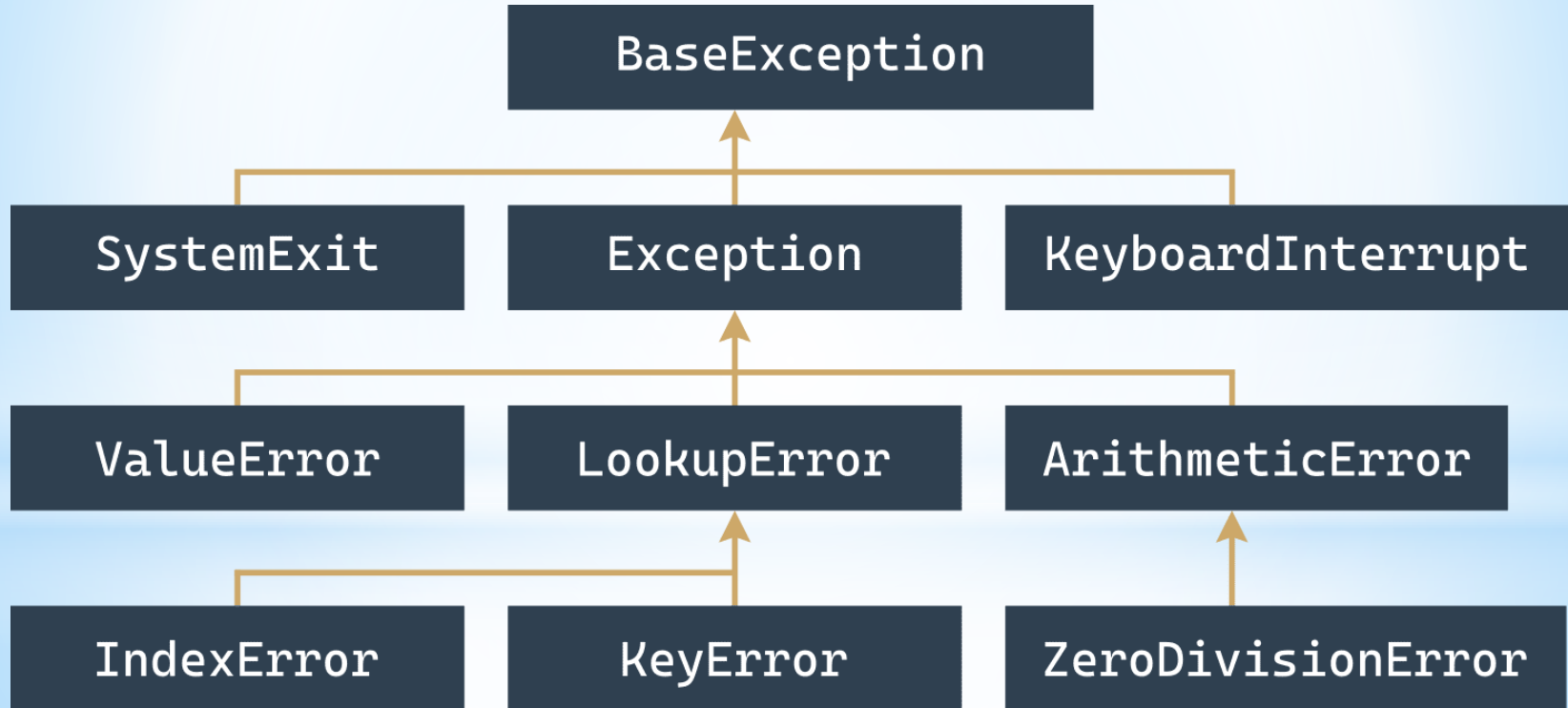
retorno del valor

```
print("Inicia aqui.")  
message()  
print("Termina aqui.")
```



JERARQUÍA DE EXCEPCIONES

<https://docs.python.org/3/library/exceptions.html>



REFERENCIAS

- * Calzi, G. & Makhoulouf, A. (2020). Introduction to Programming and Computer Programming. Curso *Python Essentials 1*, Cisco Networking Academy.
- * Cátedra Uno (2023). *Lenguajes de programación: los 3 niveles*. Junio 20, 2023. Sitio web: <https://catedrauno.com/programacion-lenguajes-niveles-436/>
- * Duque, R. (2016). *Python para todos*. España, pp. 7-15.
- * EDteam (2020). *Tipos de lenguajes de programación*. Junio 20, 2023. Sitio web: <https://ed.team/blog/tipos-de-lenguajes-de-programacion>
- * Ericson, B. & Guzdial, M. (2013). *Introducción a la computación y programación con Python*. México: Pearson Educación, pp. 16-20.
- * Garrido, J. M. (2016). *Introduction to Computational Models with Python*. USA: CRC Press, pp. 14-17.
- * Hinojosa, A. P. (2016). *Python Paso a Paso*. España: RA-MA, pp. 15-18.
- * Python Institute [OpenEDG Python Institute]. (2019, Febrero 14). *PCEP - Certified Entry-Level Python Programmer Certification* [Archivo de video]. Recuperado de <https://youtu.be/8qG463VRPjI>
- * Sandoval, L. & Solano, J. (2017). *Guía práctica de estudio 06: Entorno y fundamentos del lenguaje C*. Semestre 2018-1 (agosto-noviembre). Facultad de Ingeniería, UNAM.
- * Universidad Europea de Madrid (2010). *Análisis Semántico*. Junio 20, 2023, de Laureate International Universities. Sitio web: https://www.cartagena99.com/recursos/alumnos/apuntes/ININF2_M4_U5_T1.pdf

REFERENCIAS

- * Calzi, G. & Makhoul, A. (2020). Boolean Values, Conditional Execution, Loops, Lists and List Processing, Logical and Bitwise Operations. Curso *Python Essentials 1*, Cisco Networking Academy.
- * Chan, J. (2014). *Learn Python in One Day and Learn It Well*. USA: Wiley, pp. 49-55
- * Cuartas, M. (2020). *Iterables*. Julio 2, 2023, de Material for MkDocs. Sitio web: <http://cuartas.es/python/iterables/>
- * Garrido, J. M. (2016). *Introduction to Computational Models with Python*. USA: CRC Press, pp. 57-66.
- * Matthes, E. (2016). *Python Crash Course. A Hands-On, Project-Based Introduction to Programming*. USA: No Starch Press, pp. 117-132.
- * Moiz, S. (2022). *Cómo convertir la Int a Binario en Python*. Julio 1, 2023, de DelfStack. Sitio web: <https://www.delftstack.com/es/howto/python/python-int-to-binary/>
- * Payne, J. (2010). *Beginning Python: Using Python 2.6 and Python 3.1*. USA: Wiley, pp. 51-60.
- * Salvatierra, J. (2023). *Logical comparisons in Python: and & or*. Julio 6, 2023, The Teclado Blog. Sitio web: <https://blog.teclado.com/logical-comparisons-in-python-and-or/>
- * Solano, G. (2019). *Operadores BITWISE (AND-OR-NOT-XOR) con OpenCV y Python*. Julio 3, 2023, de OMES. Sitio web: <https://omes-va.com/operadores-bitwise/>
- * Zaczynski, B. (2021). *Bitwise Operators in Python*. Julio 2, 2023, de Real Python. Sitio web: <https://realpython.com/python-bitwise-operators/#conclusion>

REFERENCIAS

- * Calzi, G. & Makhlouf, A. (2020). *Python Data Types, Variables, Operators and Basic I/O Operations*. Curso *Python Essentials 1*, Cisco Networking Academy.
- * García, E. (2020). *¿Cómo conocer mi versión de Python?* Junio 28, 2023, de PyWombat. Sitio web: <https://pywombat.com/articles/concer-mi-version-python>
- * Garrido, J. M. (2016). *Introduction to Computational Models with Python*. USA: CRC Press, pp. 16-18.
- * Hinojosa, A. P. (2016). *Python Paso a Paso*. España: RA-MA, pp. 49-88.
- * Macías, J. (2022). *CCNAv7: Introducción a Redes - Sistemas de numeración*. Semestre 2022-1 (agosto-noviembre). Facultad de Ingeniería, UNAM.
- * Varios. (2023). *Escribir ecuaciones en LaTeX*. Junio 28, 2023, de Tutoriales, documentación y código para diseñar con Logo Latex. Sitio web: <https://manualdelatex.com/tutoriales/ecuaciones>
- * Wang, J. (2022). *Variables - C++ vs. Python*. Junio 28, 2023, de COMP70050 Autumn Term 2021/2022. Sitio web: <https://intro2ml.pages.doc.ic.ac.uk/autumn2021/modules/lab-cpp/variables>

REFERENCIAS

- * Bustamante, S. (2021). *Operadores Básicos en Python con ejemplos*. Julio 12, 2023, de freeCodeCamp. Sitio web: <https://www.freecodecamp.org/espanol/news/operadores-basicos-en-python-con-ejemplos/>
- * Calzi, G. & Makhoul, A. (2020). *Functions, Tuples, Dictionaries, Exceptions and Data Processing*. Curso Python Essentials 1, Cisco Networking Academy.
- * Chan, J. (2014). *Learn Python in One Day and Learn It Well*. USA: Wiley.
- * Duque, R. (2016). *Python para todos*. España.
- * Ericson, B. & Guzdial, M. (2013). *Introducción a la computación y programación con Python*. México: Pearson Educación.
- * Garrido, J. M. (2016). *Introduction to Computational Models with Python*. USA: CRC Press.
- * Hinojosa, A. P. (2016). *Python Paso a Paso*. España: RA-MA.
- * Matthes, E. (2016). *Python Crash Course. A Hands-On, Project-Based Introduction to Programming*. USA: No Starch Press.